

YAPL-S Lexer, Parser & IR Implementation

YAPL Extended with Native String Operations
CS 327: Compilers

Group 6

21110202 21110060 22110099 21110210

Spring 2026

Contents

1	Introduction	3
2	IR Generation Design and Implementation	3
2.1	Prerequisites to IR (Semantic Foundations)	3
2.1.1	Syntax-Directed Translation (SDT) Architecture	3
2.1.2	The Scoped Symbol Table and Archival System	4
2.1.3	Semantic Validation and Context-Aware Errors	4
2.2	IR Generation per Construct	5
2.2.1	Basic Expressions and L-Value Synthesis	5
2.2.2	Pointers, Arrays, and Type Casting	5
2.2.3	Control Flow and Logical Short-Circuiting	5
2.2.4	Function Definitions and Calls	6
2.2.5	Custom Language Features: Native YAPL-S Strings	6
2.2.6	Excluded Constructs and Architectural Constraints	6
2.3	Output Formatting and Evaluation Artifacts	7
2.3.1	Quadruple Format Specification	7
2.3.2	Benchmark Suite (20 Evaluated Artifacts)	7
2.4	Summary & Shared Artifacts	11
3	Appendix: Base YAPL Language Specification	12
3.1	Language Constructs	12
3.2	Supported Data Types	12
3.3	Supported Operators	12
3.4	Unsupported Constructs	13
4	Appendix: YAPL-S Language Specification	13
4.1	Lexical Conventions	13
4.1.1	String Concatenation Operator	13
4.1.2	Interpolated String Literals	13
4.2	Syntax	15
4.2.1	Operator Precedence and Associativity	15
4.2.2	Grammar Specification	15
4.3	Semantics	16
4.3.1	F-String Evaluation	16
4.3.2	The Concatenation Operator (@)	16
4.4	Examples and Use Cases	17
4.4.1	Initialization and Assignment	17

4.4.2	Usage in Function Calls	17
4.4.3	Interaction with Pointer Arithmetic	18
4.4.4	Nested Expressions and Escaping	18
4.4.5	Compiler Error Handling	18
5	Appendix: YAPL-S - Lexer and Parser Architecture Guide	19
5.1	Architectural Delta: YAPL vs. YAPL-S	19
5.2	YAPL-S Lexer: Context-Sensitive Tokenization	19
5.2.1	Python PEP 701: Syntactic formalization of f-strings	19
5.2.2	YAPL-S Stack-Based Lexer Implementation	20
5.2.3	The <FSTRING> State and Token Rules	20
5.3	YAPL-S Parser	21
5.3.1	The Concatenation Operator Precedence Tier	21
5.3.2	Left-Recursive String Assembly	21
5.4	Tracing an Example Nested F-String	23
5.5	Shared Artifacts	23
6	Appendix: YAPL-S Parser Improvement	24
6.1	Construction of LALR(1) Automaton	24
6.1.1	Construction of LALR(1) Item Sets	24
6.1.2	Building the LALR(1) Automaton	24
6.1.3	Conflict Identification	25
6.2	Conflict Resolution	27
6.2.1	Grammar Modifications & Architecture Overhaul	27
6.2.2	Adversarial Testing of Conflicts	28
6.3	Reverse Derivation Tree	30
6.3.1	Lexer Changes	30
6.3.2	Parser Changes	30
6.3.3	Graphviz Integration	30
6.4	LALR(1) Parsing Table	32
6.5	Error Diagnosis	32
6.6	Shared Artifacts	33

1 Introduction

YAPL-S is a variant of the YAPL language (a subset of ANSI 2011 C Standard) as taught in **CS327: Compilers** course at IITGN. It retains the core syntax and semantics of YAPL (language constructs, supported data types and supported operators), but additionally introduces native support for basic string manipulation as YAPL does not support `string` preprocessors.

This specification details the extensions made to the base YAPL language to support Pythonic **F-Strings** (Format Strings) and **String Concatenation** as native language features, eliminating the need for the `<string.h>` library for common string operations.

Note: The original YAPL and YAPL-S language specifications along with Lexer and Parser implementations and corresponding improvements are provided in Appendix 3, 4, 5 and 6

2 IR Generation Design and Implementation

In this phase, we extended our lexer-parser pipeline into a semantics-aware intermediate code generator. Our objective was not only to recognize valid YAPL-S programs, but to translate them into deterministic and inspectable Three-Address Code (3AC) quadruples that preserve source-level intent. We adopted a strict one-pass design: semantic checks, scope handling, and IR emission happen during parse-time reductions.

This design deliberately favors transparency and debuggability over aggressive optimization. Almost every construct we support has a direct lowering path into quads, which allows us to validate behavior on test artifacts without requiring a backend optimizer. We validated this implementation over 20 benchmark artifacts (10 positive, 5 negative, and 5 complex), covering arithmetic, control flow, function calls, strings, pointers, arrays, and error cases.

2.1 Prerequisites to IR (Semantic Foundations)

2.1.1 Syntax-Directed Translation (SDT) Architecture

To support IR generation in a bottom-up LALR(1) parser, we replaced AST-centric semantic transfer with a compact synthesized attribute bundle. Instead of passing only tree pointers, we propagated code-generation metadata through grammar reductions. This enabled immediate quad emission at each reduction point while preserving type information and memory-location intent.

The key architectural shift was to treat parsing and translation as one coupled process. Each nonterminal returns enough information for its parent to either (a) emit a final quad directly, or (b) build control-flow skeletons and defer only label binding. In practice, this reduced bookkeeping complexity and removed the need for a separate AST traversal pass for most constructs.

```
1 typedef struct Attributes {  
2     char place[32];  
3     int type;  
4     int pointer_depth;  
5     int is_deref;  
6     char addr_place[32];  
7     char true_label[32];  
8     char false_label[32];  
9     char next_label[32];  
10 } Attributes;
```

The fields `place` and `type` drive standard expression lowering; `pointer_depth` preserves pointer-level semantics; and `is_deref` with `addr_place` resolves l-value/r-value ambiguity for arrays and dereferences during assignment. The label slots (`true_label`, `false_label`, `next_label`) are used in branch-oriented lowering for conditionals and loops.

To maintain IR naming determinism, we generate temporaries and labels from monotonic counters:

```
1 char* newtemp() { /* returns t0, t1, ... */ }
2 char* newlabel() { /* returns L0, L1, ... */ }
```

This simple decision significantly improves reproducibility across test runs and makes diff-based artifact review practical.

2.1.2 The Scoped Symbol Table and Archival System

We implemented a spaghetti-stack symbol table using linked `Scope` objects, each holding a hash table of symbols and a parent pointer for lexical scoping. This gives us expected behavior for shadowing and outward lookup while keeping insertion local to the current scope.

Our key design choice was an archive chain for exited scopes. Instead of freeing symbols on `exit_scope()`, we detach the scope from active traversal and move it to `archive_next`. This satisfies two goals simultaneously: preserving full symbol history for the final printout requirement and avoiding accidental use-after-free while parsing nested blocks.

At lookup time, we traverse only active parent links, which keeps resolution semantics correct. At report-print time, we traverse both active and archived chains, which gives us a complete post-parse snapshot of declared symbols (including scope level, type, pointer depth, and declaration line).

```
1 typedef struct Scope {
2     int level;
3     Symbol *buckets[SYMTAB_BUCKETS];
4     struct Scope *parent;
5     struct Scope *archive_next;
6 } Scope;
7
8 void exit_scope(void) {
9     Scope *old_scope = g_current_scope;
10    g_current_scope = old_scope->parent;
11    old_scope->parent = NULL;
12    old_scope->archive_next = g_archived_scopes;
13    g_archived_scopes = old_scope;
14 }
```

This archival strategy also keeps memory ownership clear: no symbol nodes are leaked transiently during parse-time scope exits, and all allocations are safely reclaimed in `syntab_destroy()`.

2.1.3 Semantic Validation and Context-Aware Errors

IR generation without semantic validation produces misleading code. Therefore, we enforced strict checks at reduction time: redeclaration within scope, undeclared identifier use, type mismatch in assignment/operations, pointer misuse (e.g., dereference of non-pointer), and operator-specific constraints (e.g., concatenation on string-compatible operands).

We also upgraded error handling from generic parser failures to context-aware messages using `snprintf`, so users see precise semantic reasons.

```
1 if (sym == NULL) {
2     snprintf(err, sizeof(err),
3         "Semantic Error: Undeclared identifier '%s'", name);
4     yyerror(err);
5 }
```

This is particularly valuable in a one-pass compiler: once an incorrect quad is emitted, downstream traces become noisy. Early semantic rejection keeps output trustworthy and directly aligns negative test outcomes with intended failure points.

2.2 IR Generation per Construct

2.2.1 Basic Expressions and L-Value Synthesis

Arithmetic and assignment IR follow native Yacc precedence tiers, so reductions directly map into quadruples such as `+`, `-`, `*`, `/`, `%`, and `=`. We generate temporaries using `newtemp()` and emit one quad per semantic operation.

The crucial challenge was distinguishing l-values from computed r-values. For expressions like `*p = x` or `arr[i] = x`, the left side reduces through loads and address calculations. If we emit plain `=` into a temporary, we lose the memory write. We solved this by carrying `is_deref` and `addr_place` through unary/postfix rules and emitting `store` on assignment when the LHS denotes memory.

```
1 if ($1.is_deref) {
2     emit("store", $3.place, "", $1.addr_place);
3 } else {
4     emit("=", $3.place, "", $1.place);
5 }
```

This decision fixes a classic correctness bug in direct-SDT compilers: assigning to the loaded value rather than the effective address. With explicit address-carrying attributes, array and pointer writes become semantically faithful.

2.2.2 Pointers, Arrays, and Type Casting

We track pointer depth in declarations and expressions, which allows us to verify valid `&` and `*` usage and preserve resulting types. Array indexing lowers to address arithmetic plus load/store forms. We scale offsets by the referenced element width before pointer addition.

```
1 t0 = i * 4
2 t1 = base + t0
3 t2 = *t1
```

This explicit scaling ensures index semantics remain architecture-consistent at IR level. For cast expressions, we preserve the target type in attributes and emit typed conversions where required by operation context.

For pointer arithmetic, we differentiate two cases:

- **Pointer $\pm$ Integer:** legal after scaling integer offset by pointee width.
- **Pointer – Pointer:** legal only for compatible pointee types, lowered to an integer distance model.

By checking pointer depth and base type at reduction time, we block unsafe arithmetic early and avoid IR-level undefined behavior.

2.2.3 Control Flow and Logical Short-Circuiting

Bottom-up parsing cannot always "look ahead" semantically to emit branch labels at intuitive source points. We handled this with targeted mid-rule actions and label synthesis for `if`, `if-else`, `while`, `do-while`, and `for`. The resulting control-flow IR relies on `ifFalse`, `ifTrue`, `goto`, and `label`.

For short-circuiting, we implemented explicit branch-based lowering of `&&` and `||`, guaranteeing the RHS is evaluated only when required.

```
1 ifFalse a goto L_false
2 ifFalse b goto L_false
3 t0 = 1
4 goto L_end
5 L_false:
```

```

6 t0 = 0
7 L_end:

```

To support `break` and `continue` across nested loops, we introduced global loop tracking stacks (`loop_start_stack`, `loop_exit_stack`) indexed by current loop depth.

This stack-based loop metadata is essential for nested constructs. A `continue` always jumps to the innermost loop’s continuation target, while `break` jumps to the corresponding exit label. By binding these at loop-entry and popping on loop-exit, we maintain correct dynamic control transfer without AST backpatch arrays.

2.2.4 Function Definitions and Calls

Function calls lower into argument pushes plus call quads: `param`, then `call f, N, t`. Return statements emit `return` with or without an operand. Function bodies start with a `label` quad.

One important and frustrating bug appeared in function symbol insertion: parameter parsing could overwrite global `current_decl_type`, causing the wrong return type to be stored for the function symbol. We fixed this by capturing the function return type from `declaration_specifiers` attributes and using that captured value in `function_definition`, rather than reading mutable global state at insertion time.

```

1 function_definition
2   : declaration_specifiers declarator
3     {
4         insert_symbol($2.place, $1.type, $2.pointer_depth, yylineno);
5         emit("label", "", "", $2.place);
6     }
7     compound_statement

```

Argument evaluation order is emitted left-to-right through `argument_expression_list`, with each argument producing a `param` quad. The call quad stores both callee name and argument count, which keeps call boundaries explicit for downstream passes.

```

1 param arg0
2 param arg1
3 call foo 2 t7

```

2.2.5 Custom Language Features: Native YAPL-S Strings

We treat strings as `char *` semantically, and we introduced specialized IR operations for string construction and interpolation. Concatenation uses `concat`, while f-string lowering uses `append_str` for static chunks and `append_var` for interpolated expressions.

```

1 t0 = ""
2 append_str "[" t0
3 append_var module t0
4 append_str "]" t0
5 append_str " Failed with code: " t0
6 append_var code t0

```

This IR abstracts runtime buffer management and keeps parser actions concise and deterministic.

This abstraction is intentional: memory allocation and formatting policy can evolve independently in a future runtime layer, while frontend semantics remain stable. In other words, our frontend emits *what* string operations must happen, not *how* heap operations are micro-managed.

2.2.6 Excluded Constructs and Architectural Constraints

We intentionally left out some constructs from full IR lowering in this stage due to one-pass architectural limits:

- **Full struct field layout:** complete support needs byte-offset computation and richer type environment propagation; this is significantly easier with a dedicated AST/type-check pass.
- **Robust switch/jump table lowering:** efficient lowering requires forward target collection and case-table planning before final branch emission.
- **Multi-dimensional arrays:** this needs dimension-bound metadata and row-major offset synthesis over multiple indices.

In short, each omitted construct is feasible, but implementing it cleanly in strict one-pass mode increases grammar-side state complexity and conflict risk.

If we move to a two-pass architecture (build AST/type graph first, then emit IR), these exclusions become significantly easier to support while keeping grammar actions minimal and conflict-resistant.

2.3 Output Formatting and Evaluation Artifacts

2.3.1 Quadruple Format Specification

We standardized IR output in a strict four-field notation:

(Op, Arg1, Arg2, Result).

Each emitted instruction is printed in tabular sequence order, which makes dataflow and control flow directly traceable during evaluation.

#	Op	Arg1	Arg2	Result
0	label			main
1	=	500		code
2	param	t2		
3	call	log_event	1	t3
4	return	status		

We print quads in the exact emission order. This ordering is crucial for validation because it preserves control-flow structure and temporary dependencies. During debugging, we can directly map questionable behavior back to the reduction that emitted the corresponding quad.

2.3.2 Benchmark Suite (20 Evaluated Artifacts)

To validate our implementation, we organized tests into three buckets under `tests/ir`:

- **Positive (10):** valid programs that must parse and generate correct IR.
- **Negative (5):** semantically invalid programs that must fail with informative errors.
- **Complex (5):** integrated stress cases combining functions, loops, pointers, arrays, and string features.

Each test has a source artifact and a generated IR output artifact (plus parse tree visualizations where applicable), giving us reproducible evidence for successful IR-stage implementation. Below we present 11 representative cases with output excerpts.

Positive Case 1: Basic Arithmetic and Precedence

Source (`tests/ir/positive/test1.c`):

```

1 int main() {
2     int a; int b; int c;
3     a = 10; b = 20;
4     c = a + b * 2 - (a / 5);
5     return c;
6 }
```

Output excerpt (tests/ir/positive/output_ir_new_1.txt):

```
1 ===== 3AC QUADRUPLES =====
2 #      Op      Arg1      Arg2      Result
3 3      *      b      2      t0
4 4      +      a      t0      t1
5 5      /      a      5      t2
6 6      -      t1      t2      t3
7 7      =      t3      c
8 8      return   c
9 ***parsing successful***
```

Positive Case 2: Type Cast Lowering

Source (tests/ir/positive/test2.c):

```
1 int main() {
2     float pi; int approx;
3     pi = 3.14;
4     approx = (int) pi;
5     return approx;
6 }
```

Output excerpt (tests/ir/positive/output_ir_new_2.txt):

```
1 ===== 3AC QUADRUPLES =====
2 #      Op      Arg1      Arg2      Result
3 1      =      3.14      pi      pi
4 2      cast    pi      Int      t0
5 3      =      t0      approx
6 4      return   approx
7 ***parsing successful***
```

Positive Case 3: Function Call and Return

Source (tests/ir/positive/test6.c):

```
1 int add(int a, int b) {
2     return a + b;
3 }
4 int main() {
5     int result;
6     result = add(5, 10);
7     return result;
8 }
```

Output excerpt (tests/ir/positive/output_ir_new_6.txt):

```
1 ===== 3AC QUADRUPLES =====
2 #      Op      Arg1      Arg2      Result
3 0      label    add
4 1      +      a      b      t0
5 2      return   t0
6 4      param    5
7 5      param    10
8 6      call     add      2      t1
9 7      =      t1      result
10 ***parsing successful***
```

Positive Case 4: String Concatenation (@)

Source (tests/ir/positive/test9.c):

```
1 int main() {
2     char *prefix; char *suffix; char *msg;
3     prefix = "Error: ";
```

```

4     suffix = "Out of bounds";
5     msg = prefix @ suffix;
6     return 0;
7 }

```

Output excerpt (tests/ir/positive/output_ir_new_9.txt):

```

1 ===== 3AC QUADRUPLES =====
2 #      Op          Arg1          Arg2          Result
3 1      =           "Error: "      prefix
4 2      =           "Out of bounds" suffix
5 3      concat      prefix          suffix      t0
6 4      =           t0              msg
7 5      return      0
8 ***parsing successful***

```

Positive Case 5: F-String Interpolation

Source (tests/ir/positive/test10.c):

```

1 int main() {
2     int user_id; int attempts; char *log;
3     user_id = 404;
4     attempts = 3;
5     log = f"User {user_id} failed {attempts} times.";
6     return 0;
7 }

```

Output excerpt (tests/ir/positive/output_ir_new_10.txt):

```

1 ===== 3AC QUADRUPLES =====
2 #      Op          Arg1          Arg2          Result
3 3      =           ""              t0
4 4      append_str   User              t0
5 5      append_var   user_id           t0
6 6      append_str   failed            t0
7 7      append_var   attempts          t0
8 8      append_str   times.            t0
9 9      =           t0              log
10 ***parsing successful***

```

Negative Case 1: Undeclared Identifier

Source (tests/ir/negative/test1.c):

```

1 int main() {
2     limit = 100;
3     return 0;
4 }

```

Output excerpt (tests/ir/negative/output_ir_new_1.txt):

```

1 Location : Line 3, Column 11
2 Token    : '='
3 Message  : Semantic Error: Undeclared identifier 'limit'
4 ***parsing terminated*** [syntax error]

```

Negative Case 2: Type Mismatch in Assignment

Source (tests/ir/negative/test3.c):

```

1 int main() {
2     int x; char *str;
3     str = "Hello";
4     x = str;
5     return 0;
6 }

```

Output excerpt (tests/ir/negative/output_ir_new_3.txt):

```
1 Location : Line 5, Column 12
2 Token    : ';'
3 Message   : Semantic Error: Type mismatch in assignment
4 ***parsing terminated*** [syntax error]
```

Negative Case 3: Invalid Pointer Dereference

Source (tests/ir/negative/test4.c):

```
1 int main() {
2     int num; int val;
3     num = 5;
4     val = *num;
5     return 0;
6 }
```

Output excerpt (tests/ir/negative/output_ir_new_4.txt):

```
1 Location : Line 5, Column 15
2 Token    : ';'
3 Message   : Semantic Error: Cannot dereference a non-pointer
4 ***parsing terminated*** [syntax error]
```

Complex Case 1: Deep Scoping and Shadowing

Source (tests/ir/complex/test1.c):

```
1 int main() {
2     int x; x = 1;
3     {
4         int x; x = 2;
5         {
6             int x; x = 3;
7         }
8     }
9     return x;
10 }
```

Output excerpt (tests/ir/complex/output_ir_new_1.txt):

```
1 ===== 3AC QUADRUPLES =====
2 #      Op          Arg1          Arg2          Result
3 1      =            1              x
4 2      =            2              x
5 3      =            3              x
6 4      return      x
7 ***parsing successful***
```

Complex Case 2: Nested Loops with Break/Continue

Source (tests/ir/complex/test3.c):

```
1 int main() {
2     int i; int j; int sum;
3     sum = 0;
4     for (i = 0; i < 3; i = i + 1) {
5         j = 0;
6         while (j < 5) {
7             if (j == 2) { break; }
8             sum = sum + 1;
9             j = j + 1;
10        }
11        if (i == 1) { continue; }
12        sum = sum + 10;
13    }
```

```

13     }
14     return sum;
15 }

```

Output excerpt (tests/ir/complex/output_ir_new_3.txt):

```

1 ===== 3AC QUADRUPLES =====
2 #      Op      Arg1      Arg2      Result
3      label                                L0
4      ifFalse      t0      goto      L1
5 13      label                                L4
6 15      ifFalse      t2      goto      L5
7 18      goto                                L5
8 28      goto                                L3
9 34      return      sum
10 ***parsing successful***

```

Complex Case 3: Combined F-String + Concatenation + Function Call

Source (tests/ir/complex/test5.c):

```

1 int log_event(char *message) {
2     return 1;
3 }
4
5 int main() {
6     int code; char *module; int status;
7     code = 500;
8     module = "Auth";
9     status = log_event(f"[{module}]" @ f" Failed with code: {code}");
10    return status;
11 }

```

Output excerpt (tests/ir/complex/output_ir_new_5.txt):

```

1 ===== 3AC QUADRUPLES =====
2 #      Op      Arg1      Arg2      Result
3      =      ""                                t0
4      append_str      [                                t0
5      append_var      module                                t0
6      append_str      ]                                t0
7      =      ""                                t1
8 10      append_str      Failed with code:      t1
9 11      append_var      code                                t1
10 12      concat      t0      t1      t2
11 13      param      t2
12 14      call      log_event      1      t3
13 ***parsing successful***

```

The positive suite checks semantic correctness and expected quad structure; the negative suite verifies strict rejection behavior and diagnostic quality; and the complex suite validates compositional correctness under combined features (e.g., function calls inside loops with string interpolation and pointer-sensitive assignments).

2.4 Summary & Shared Artifacts

Through this phase, we transformed YAPL-S from a language recognizer into a semantically checked IR producer with explicit treatment of memory, control flow, function boundaries, and string abstractions. Our implementation choices prioritized deterministic one-pass generation, clear diagnostics, and evaluable output formatting.

Most importantly, the final system is coherent end-to-end: scoped symbols feed semantic checks, semantic attributes feed IR emission, and emitted quads feed reproducible artifact validation. The benchmark outcomes demonstrate that our quadruples are stable, readable, and faithful to language semantics across simple, erroneous, and composition-heavy programs.

You can find the source code of here: [Github Repo](#)

3 Appendix: Base YAPL Language Specification

This appendix details the specification for the base YAPL language (a subset of C11) as available here: [YAPL Language Specification \(PDF\)](#).

3.1 Language Constructs

The language conforms to the ANSI C-2011 standard with the following specific inclusions:

- global variables, both normal as well as **extern**.
- function declaration, function definition, and function call (including recursion).
- **for** loop, **while** loop, **do-while**.
- **if**, **if-else**, **if-else-if** (nested/ladder) (if condition can be any expression including literals/constants).
- **switch/case** with case labeled with **::**; must support fall-through; must support the optional **default** clause.
- variable declaration, definition, initialization locally/globally (multiple variables comma-separated).
- strings (of the forms `"..."` (single), `"..." "..."` (multiple)), character, integer and floating point literals.
- single-line (`//`) and multi-line (`/*...*/`) comments (multi-line comments should be properly closed lexically).
- **break**, **continue**, and **return** statements.
- a statement must end in a semi-colon `;` like the usual standard of C.

3.2 Supported Data Types

- **Primary:** `int`, `long`, `short`, `float`, `double`, `void`, `char` and their pointers.
- **User-defined:** structures (**struct**) (pointer dereferences must support arbitrary level of indirection).
- **Derived:** functions, arrays, and pointers (all supported data types; no need to support function pointers).

3.3 Supported Operators

Operator precedence and associativity follow standard C rules.

- **Relational operators:** `<`, `>`, `==`, `<=`, `>=`, `!=`, `<=>`.
- **Unary operators:** `+`, `-`, `!`, `*`, `&`, `~`.
- **Binary operators:** `+`, `-`, `*`, `&`, `|`, `/`, `%`, `^`.
- **Logical operators:** `&&`, `||`.

- **Assignment operator:** =.
- **Suffix/postfix increment and decrement:** ++, --.
- **Structure field access operators:** ->, ..
- **Pointers:** *, & (pointer dereferences must support arbitrary/multiple level of indirection, i.e., `int *****p;`).
- **Function call:** () (any valid expression inside including blank, constants, and literals).
- **Array subscripting:** [] (any valid expression inside including constants and literals).
- **sizeof():** operands: <typename> or unary expression.
- **Typecast:** (<typename>).

3.4 Unsupported Constructs

The following C11 features are **explicitly excluded**:

- **Preprocessors:** none allowed (no `#includes`, `#defines` needed anywhere).
- **Operators:** ... (ellipsis), ?: (ternary operator) (function vararg list specification not required).
- **Operators:** >>=, <<=, +=, -=, *=, /=, %=, &=, ^=, |=, <<, >>.
- **Constructs/keywords:** `auto`, `const`, `goto`, `inline`, `register`, `restrict`, `signed`, `static`, `typedef`, `unsigned`, `enum`, `union`, `volatile`, `_Alignas`, `_Alignof`, `_Atomic`, `_Bool`, `_Complex`, `_Generic`, `_Imaginary`, `_Noreturn`, `_Static_assert`, `_Thread_local`, `__func__`.

4 Appendix: YAPL-S Language Specification

4.1 Lexical Conventions

The lexical analyzer distinguishes tokens based on the standard C rules, with the following additions for handling string interpolation.

4.1.1 String Concatenation Operator

The following operator is added to the language:

Operator	Symbol	Description
Concatenation	@	Binary operator for joining two string literals.

Table 1: New Operators in YAPL-S

4.1.2 Interpolated String Literals

An interpolated string literal (Pythonic F-String) is distinguished from a standard string literal by the prefix `f`.

Syntax:

```
1 f" text { expression } text "
```

The lexical analyzer would process F-Strings using the `lex` tool to emit the following distinct token types:

- `FSTRING_START`: The sequence `f`.
- `STRING_LITERAL_PART`: A sequence of characters inside the F-String that are *not* part of an interpolation.
- `INTERPOLATION_START`: The `{` character inside an F-String.
- `INTERPOLATION_END`: The `}` character inside an F-String.
- `FSTRING_END`: The closing `"` character.

Note: Standard escape sequences (e.g., `\n`, `\n`) are valid within `STRING_LITERAL_PART`.

4.2 Syntax

The grammar of YAPL-S extends the YAPL grammar.

4.2.1 Operator Precedence and Associativity

The @ operator is inserted strictly between the **Shift** and **Relational** tiers. This ensures `char` * arithmetic (addition/subtraction) takes priority over concatenation.

Prec.	Operator Class	Operators	Associativity
1	Primary/Postfix	() [] -> .	L-to-R
2	Unary	* & - !	R-to-L
3	Multiplicative	* / %	L-to-R
4	Additive	+ -	L-to-R
5	Shift	<< >>	L-to-R
6	Concatenation	@	L-to-R
7	Relational	< > <= >= <=>	L-to-R
8	Equality	== !=	L-to-R

Table 2: Revised Operator Precedence Table

4.2.2 Grammar Specification

The grammar hierarchy reflects the precedence table.

```
1 // 1. Relational expressions derive from concatenation
2 relational_expression
3   : concatenation_expression
4   | relational_expression '<' concatenation_expression
5   | relational_expression '>' concatenation_expression
6   | relational_expression LE_OP concatenation_expression
7   | relational_expression GE_OP concatenation_expression
8   | relational_expression TH_OP concatenation_expression
9   ;
10
11 // 2. Concatenation derives from Shift
12 // This ensures @ has lower precedence than <<, +, *, etc.
13 concatenation_expression
14   : shift_expression
15   | concatenation_expression '@' shift_expression
16   ;
17
18 // 3. Shift derives from Additive (Standard C)
19 shift_expression
20   : additive_expression
21   | shift_expression LEFT_OP additive_expression
22   | shift_expression RIGHT_OP additive_expression
23   ;
24
25 // 4. Primary Expression updated for F-Strings
26 primary_expression
27   : IDENTIFIER
28   | constant
29   | string
30   | '(' expression ')'
31   | generic_selection
```

```

32 | f_string_expression /* Extension */
33 ;
34
35 // 5. F-String Structure
36 f_string_expression
37 : FSTRING_START f_string_body FSTRING_END
38 ;
39
40 f_string_body
41 : /* empty */
42 | f_string_body STRING_LITERAL_PART
43 | f_string_body interpolation_block
44 ;
45
46 interpolation_block
47 : INTERPOLATION_START expression INTERPOLATION_END
48 ;

```

4.3 Semantics

4.3.1 F-String Evaluation

An F-String expression evaluates to a value of type `char *`.

1. **Evaluation Order:** The expressions inside the interpolation blocks $\{\dots\}$ are evaluated from left to right at runtime.
2. **Type Conversion:** The result of each interpolated expression is implicitly converted to its string representation.
 - **int:** Converted to decimal string (e.g., $10 \rightarrow "10"$).
 - **float/double:** Converted to standard floating-point representation.
 - **char *:** Copied as-is.
 - **char:** Converted to a single-character string.
3. **Construction:** The literal text parts and the converted expression strings are concatenated in order.
4. **Memory:** The result is stored in a newly allocated memory block (heap). The runtime environment manages this allocation.

4.3.2 The Concatenation Operator (@)

The @ operator performs string concatenation.

- **Operands:** Both operands must evaluate to type `char *`.
- **Result:** A new `char *` containing the contents of the left operand followed immediately by the right operand.
- **Error Handling:** The compiler shall issue a Type Error if an operand is numeric (e.g., `str @ 5` is illegal).
- **Behavior:** Resultant string is automatically null-terminated.
 1. Compute length of left string (L_1) and right string (L_2).
 2. Allocate $L_1 + L_2 + 1$ bytes on the heap.
 3. Copy Left String $\rightarrow$ Copy Right String $\rightarrow$ Null Terminate.

- **Rationale for Precedence:** Given `str1 @ str2 + 1`:

- Additive (+) runs first: `str2 + 1` advances the pointer by 1 char.
- Concatenation (@) runs second: Joins `str1` with the substring.
- This prevents the inefficiency of allocating a new string only to immediately skip its first character.

4.4 Examples and Use Cases

4.4.1 Initialization and Assignment

Native strings can be created dynamically without reliance on `sprintf` or `strcat`.

```
1 int main() {
2     int id = 42;
3     double score = 98.5;
4     char *user = "Alice";
5
6     // --- Standard C11 Approach (Tedious) ---
7     // Requires <stdio.h>, <stdlib.h>, <string.h>
8     // char buffer[100];
9     // sprintf(buffer, "User: %s (ID: %d)", user, id);
10
11    // --- YAPL-S Approach (Native) ---
12    // F-Strings handle implicit memory allocation and formatting
13    char *status = f"User: {user} (ID: {id})";
14
15    // String Concatenation using the @ operator
16    char *full_msg = status @ " - Status: Active";
17
18    // full_msg is now: "User: Alice (ID: 42) - Status: Active"
19
20    return 0;
21 }
```

4.4.2 Usage in Function Calls

F-Strings evaluate to `char *`, allowing them to be passed directly to any function expecting a C string.

```
1 void log_event(char *msg) {
2     // Implementation for logging...
3 }
4
5 int main() {
6     int x = 10, y = 20;
7
8     // Direct usage in printf (replaces %s format specifiers)
9     printf(f"Coordinates: x={x}, y={y}\n");
10
11    // Usage in custom functions with embedded arithmetic
12    // The expression {x + y} is evaluated before string construction
13    log_event(f"System Check: {x + y} units detected.");
14
15    return 0;
16 }
```

4.4.3 Interaction with Pointer Arithmetic

Because `@` has lower precedence than additive operators (`+`, `-`), standard C pointer arithmetic behaves predictably when mixed with concatenation.

```
1 int main() {
2     char *h = "Hello";
3     char *w = "World";
4
5     // Scenario A: Pointer Arithmetic on the Operand (Standard
6     // Precedence)
7     // 1. ("World" + 1) advances pointer to "orld"
8     // 2. "Hello" @ "orld" concatenates
9     char *subset = h @ w + 1;
10    // Result: "Helloorld"
11
12    // Scenario B: Pointer Arithmetic on the Result (Grouped)
13    // 1. ("Hello" @ "World") creates "HelloWorld"
14    // 2. (+ 1) advances pointer on the new string
15    char *offset = (h @ w) + 1;
16    // Result: "elloWorld"
17
18    return 0;
19 }
```

4.4.4 Nested Expressions and Escaping

Based on the implementation, F-strings can be nested arbitrarily deep, enabling complex formatting logic.

```
1 int main() {
2     int val = 100;
3
4     // Nested F-String:
5     // The inner expression { f"[{val}]" } evaluates to "[100]" first.
6     char *debug = f"Debug Info: { f"Value -> [{val}]" }";
7
8     return 0;
9 }
```

4.4.5 Compiler Error Handling

The compiler enforces strict type safety for the `@` operator to prevent accidental pointer miscalculations.

```
1 int main() {
2     char *base = "Error Code: ";
3     int code = 404;
4
5     // INVALID: Cannot concatenate string (char *) with integer (int)
6     // char *err = base @ code; <-- Compilation Error
7
8     // CORRECT: Use F-String to convert the integer first
9     char *msg = base @ f"{code}";
10    // Result: "Error Code: 404"
11
12    return 0;
13 }
```

5 Appendix: YAPL-S - Lexer and Parser Architecture Guide

In this document, we will go through the architectural additions and internal mechanisms used to extend the base YAPL compiler into YAPL-S. It assumes familiarity with the YAPL-S language specification and focuses on how the new features were implemented in Lex (`yapl_s.l`) and Yacc (`yapl_s.y`), with a special emphasis on the **state-based tokenizer**.

5.1 Architectural Delta: YAPL vs. YAPL-S

The base YAPL language is a standard subset of C11 that lacks native string manipulation capabilities. YAPL-S introduces two major semantic features requiring deep modifications to both the lexer and the parser:

- **Binary String Concatenation (@):**
 - **Lexer Addition:** A simple pass-through token for @.
 - **Parser Addition:** A new `concatenation_expression` tier was injected strictly between `shift_expression` and `relational_expression`. This specifically preserves standard C pointer arithmetic precedence (e.g., `ptr + 1` happens before concatenation).
- **Interpolated F-Strings (f"..."):**
 - **Lexer Addition:** A multi-state, stack-based tokenization logic to handle *context-sensitive scanning*.
 - **Parser Addition:** A left-recursive grammar rule that combines text literals and C-expressions.

5.2 YAPL-S Lexer: Context-Sensitive Tokenization

Standard Lex is a **Finite State Automaton (FSA)**. FSAs have no memory and cannot balance nested structures.

If we parse an F-string like `f"Value:{x + 1}"`, the lexer encounters a severe conflict:

- In String (dubbed String/FSTRING Mode hereafter), a space is a literal character that must be kept. The word `int` is just the letters i-n-t.
- Inside the `{}` (dubbed Code/INITIAL Mode hereafter), a space is whitespace to be ignored. The word `int` is a language keyword.

Because Lex operates ahead of the Yacc parser, Yacc cannot tell Lex how to interpret the characters. The Lexer itself must maintain its own context. This led us to *Context-Sensitive Lexing*.

5.2.1 Python PEP 701: Syntactic formalization of f-strings

The problem of nested interpolation is not unique to YAPL-S. Python faced this exact limitation. Before Python 3.12, the CPython lexer used a "hack" where it read the entire `f"..."` as a single, massive string token, deferring the extraction of the inner `{...}` expressions to the parser. This meant you could not easily nest f-strings or use the same quote types inside them.

In PEP 701, Python completely overhauled its lexer to use a State Stack.

"This PEP proposes formalizing the f-string syntax... The parser will be modified to understand f-strings natively. This is achieved by adding a new state to the tokenizer... When the tokenizer finds an f-string, it enters a specific mode... When it finds a brace `{` inside an f-string, it drops back to the regular mode, but keeps track of the fact that it was inside an f-string using a stack." - Python PEP 701

YAPL-S adopts this exact modern architecture, effectively bypassing the limitations of older string interpolation designs.

5.2.2 YAPL-S Stack-Based Lexer Implementation

To replicate the PEP 701 behavior in standard Lex, the YAPL-S lexer creates a simulated *Pushdown Automaton (PDA)* using a C-array stack.

Core Components in `yapl_s.l`:

- **State Stack:** An array `int state_stack[100]` with `push_state()` and `pop_state()` functions. This remembers the "modes" the lexer was in.
- **Added New Start Condition (`%x FSTRING`):** An exclusive lexical state. When active, normal C code rules (like keywords or ignoring whitespace) are completely disabled. Used to scan string literals as intended.
- **Brace Counter (`brace_depth`):** An integer that counts nested `{` and `}` while in normal C Code Mode.

How the Modes Switch:

- **Entering:** When Lex sees `f"`, it saves its current state to the stack and executes `BEGIN(FSTRING)`. Now, it tokenizes string literals as is.
- **Interpolating:** When Lex sees `{` inside an F-string, it pushes `FSTRING` to the stack, resets `brace_depth = 0`, and executes `BEGIN(INITIAL)`. Now, it perfectly tokenizes C code.
- **Exiting:** When Lex sees `}` in `INITIAL` mode, it checks `brace_depth`. If `brace_depth == 0` and the top of the stack is `FSTRING`, it knows this brace closes an interpolation, not a C Code block. It pops the stack and executes `BEGIN(FSTRING)`.

5.2.3 The `<FSTRING>` State and Token Rules

To safely parse string contents without accidentally triggering standard C keywords (like `if` or `while`), the lexer defines an exclusive start condition via `%x FSTRING`. When the lexer enters this state, it ignores all normal C rules and operates inside a strict "string sandbox."

Within this sandbox, the lexer evaluates a highly specific set of rules to break the string down into its component tokens:

- **Raw Text Consumption:** The most important regular expression of the String Mode is `([~"{}\\n]|{ES})+`. It rapidly gobbles up continuous blocks of standard text and valid C-escape sequences (like `\n` or `\t`), stopping only when it hits a boundary character: a quote, a brace, a newline, or an invalid backslash.
- **Escaping Literal Braces:** YAPL-S mirrors modern standards (like Python) by using brace doubling. If the lexer encounters `{{` or `}}` while inside the `<FSTRING>` state, it explicitly matches them via the `"{{"|"}}"` rule and emits them as safe `STRING_LITERAL_PART` tokens. This prevents the lexer from falling into `INITIAL` mode, allowing programmers to safely write strings like:

```
1 char *set = f"Math Set: {{1, 2, {dynamic_val}}}"
```

- **Interpolation Entry:** If a single `{` is encountered, the sandbox pauses. The lexer pushes the `FSTRING` state to the stack, sets `brace_depth = 0`, and executes `BEGIN(INITIAL)`. This hands control back to the standard C-lexer rules, allowing the inner expression to be tokenized normally. It simultaneously emits an `INTERPOLATION_START` token for the parser.

- **String Termination:** When the lexer encounters an unescaped `"`, it knows the current string segment is complete. It pops the previous state from the stack (using `pop_state()`), executes a `BEGIN` to return to that state, and emits `FSTRING_END`.
- **Strict Error Handling:** To prevent runaway parsing, the sandbox contains strict error-catching rules:
 - **Unescaped Newlines:** F-strings, like standard C-strings, are not allowed to silently span multiple lines. Catching a raw `\n` triggers a `[lexer error]: unescaped newline in f-string`.
 - **Invalid Escapes:** A stray backslash (`\`) that does not form a valid escape sequence instantly halts the lexer with an `invalid escape sequence` error.

5.3 YAPL-S Parser

To support the new string features without breaking standard ANSI C11 behavior, the YAPL-S Yacc grammar required two major architectural injections.

5.3.1 The Concatenation Operator Precedence Tier

The base YAPL language groups operators into strictly ordered tiers (e.g., `multiplicative_expression`, `additive_expression`, `shift_expression`).

To safely introduce the `@` operator, a new `concatenation_expression` tier was inserted directly between `shift_expression` and `relational_expression`.

```

1 concatenation_expression
2   : shift_expression
3   | concatenation_expression '@' shift_expression
4   ;

```

By making concatenation derive from shift (and ultimately additive) expressions, the parser intrinsically guarantees that C pointer arithmetic (like `str + 1`) resolves before string concatenation.

5.3.2 Left-Recursive String Assembly

When the lexer breaks an F-string into chunks, the parser must stitch them back together dynamically. This is achieved using a left-recursive grammar rule for `f_string_body`.

```

1 f_string_body
2   : /* empty */
3   | f_string_body STRING_LITERAL_PART
4   | f_string_body interpolation_block
5   ;

```

Left recursion is highly efficient for LALR parsers like Yacc. It ensures the parser never has more than two or three string chunks on its internal stack at any given time. As each new `STRING_LITERAL_PART` or `interpolation_block` arrives, the parser immediately reduces it into the growing `f_string_body` node.

Note: Refer to the YAPL-S language specifications as provided in Appendix 4 to view all the changes in the grammar.

Input	Lexer Mode (Before → After)	Lexer Stack (After Push/Pop)	Token Emitted	Parser Action
f"	INITIAL → FSTRING	[INITIAL]	FSTRING_START	Shift to Parse Stack.
A_	FSTRING	[INITIAL]	STRING_LITERAL_PART	Shift then Reduce against empty <code>f_string.body</code> to start building.
{	FSTRING → INITIAL	[INITIAL, FSTRING]	INTERPOLATION_START	Shift . We are now parsing C code.
_	INITIAL	[INITIAL, FSTRING]	<i>(Ignored)</i>	Lexer discards whitespace in code mode.
f"	INITIAL → FSTRING	[INITIAL, FSTRING, INITIAL]	FSTRING_START	Shift . Starting the first inner F-string.
{	FSTRING → INITIAL	[INITIAL, FSTRING, INITIAL, FSTRING]	INTERPOLATION_START	Shift . Preparing to parse the inner variable.
b	INITIAL	[INITIAL, FSTRING, INITIAL, FSTRING]	IDENTIFIER	Shift then Reduce <code>ID</code> → <code>primary_expr</code> → <code>expr</code> .
}	INITIAL → FSTRING	[INITIAL, FSTRING, INITIAL]	INTERPOLATION_END	Shift then Reduce the inner <code>interpolation_block</code> .
"	FSTRING → INITIAL	[INITIAL, FSTRING]	FSTRING_END	Shift then Reduce the entire inner <code>f_string-expression</code> .
_	INITIAL	[INITIAL, FSTRING]	<i>(Ignored)</i>	Lexer discards whitespace again.
}	INITIAL → FSTRING	[INITIAL]	INTERPOLATION_END	Shift then Reduce the first outer <code>interpolation_block</code> and append it.
_	FSTRING	[INITIAL]	STRING_LITERAL_PART	Shift then Reduce to append the space between the blocks.
{	FSTRING → INITIAL	[INITIAL, FSTRING]	INTERPOLATION_START	Shift . Opening the second outer interpolation block.
_	INITIAL	[INITIAL, FSTRING]	<i>(Ignored)</i>	Whitespace in code mode.
f"	INITIAL → FSTRING	[INITIAL, FSTRING, INITIAL]	FSTRING_START	Shift . Starting the second inner F-string.
C	FSTRING	[INITIAL, FSTRING, INITIAL]	STRING_LITERAL_PART	Shift then Reduce to an inner <code>f_string.body</code> .
"	FSTRING → INITIAL	[INITIAL, FSTRING]	FSTRING_END	Shift then Reduce to an <code>f_string-expression</code> .
}	INITIAL → FSTRING	[INITIAL]	INTERPOLATION_END	Shift then Reduce the second outer <code>interpolation_block</code> and append.
"	FSTRING → INITIAL	[] (Empty)	FSTRING_END	Shift . The outer F-string is complete.

Table 3: Step-by-Step Lexer and Parser Trace for Nested F-String Evaluation

5.4 Tracing an Example Nested F-String

Let's trace a highly complex, deeply nested edge case: `f"A_{f"{b}"}_{f"C"}"`, where `_` denotes a space (for better readability).

This string contains an outer F-string, which contains two interpolation blocks. The first interpolation block contains another F-string, which itself interpolates a variable `b`. Furthermore, there is intentional spacing inside the code blocks which the lexer must know to ignore.

Refer to Table 3 for a step-by-step trace of how the lexer and parser collaborate to evaluate this sequence.

5.5 Shared Artifacts

Along with this document, we have attached the YAPL-S Lex (`yapl_s.l`) and Yacc (`yapl_s.y`) file. Using the `Makefile`, one can generate the lexers and parsers of YAPL-S and start tokenizing YAPL-S codes. Details of running the same are given in a dedicated `README.md` file.

We have also provided a test suite which contains both **positive** (cases where YAPL-S must successfully parse the code) and **negative** (cases where YAPL-S must fail parsing/lexing gracefully) examples. The outputs of the tokenizer along with some important grammar reductions are demonstrated as debug output in the corresponding `.txt` files.

The same can be found here: [Github Repo](#)

6 Appendix: YAPL-S Parser Improvement

6.1 Construction of LALR(1) Automaton

Given the sheer scale of a complete C grammar (which typically generates hundreds of states), constructing the entire LALR(1) automaton by hand is computationally infeasible. Instead, we leveraged `yacc` with the verbose flag (`yacc -v -d yapl.s.y`) to automatically compute the LALR(1) item sets, generate the state machine, and identify parsing conflicts. The resulting automaton and its diagnostics are captured in the generated `yapl.s.output` file.

6.1.1 Construction of LALR(1) Item Sets

An item set represents a specific state in the parser. It contains the grammar productions currently being tracked, with a dot (`.` or `.`) indicating the parser's current position within that production (i.e., what has been seen so far versus what is expected next).

For example, observing State 153 (related to newly added F-Strings) from our generated `yapl.s.output` file:

```
1 State 153
2
3     7 f_string_expression: FSTRING_START f_string_body . FSTRING_END
4     9 f_string_body: f_string_body . STRING_LITERAL_PART
5    10                | f_string_body . interpolation_block
```

In this item set, the parser has successfully matched the beginning of an F-String (`FSTRING_START`) and some arbitrary amount of `f_string_body`. The dot (`.`) indicates that the parser is currently inside the F-String and is waiting to evaluate the next token to determine if the string is ending, continuing with literal text, or opening an interpolation block.

6.1.2 Building the LALR(1) Automaton

The LALR(1) automaton is a directed graph where the item sets act as nodes (states) and the lookahead tokens act as the directed edges (transitions). The transition function is represented by the shift, reduce, and goto actions detailed in the `yapl.s.output` file.

Continuing with our F-String example in State 153, the automaton defines the following transitions based on the next input token:

```
1     FSTRING_END          shift, and go to state 262
2     INTERPOLATION_START  shift, and go to state 263
3     STRING_LITERAL_PART  shift, and go to state 264
4
5     interpolation_block   go to state 265
```

This indicates three distinct directed terminal edges (shifts) and one non-terminal edge (goto) originating from this state:

1. If the lookahead is `FSTRING_END`, the parser shifts and transitions to State 262, preparing to finalize the `f_string_expression`.
2. If the lookahead is `INTERPOLATION_START` (our `{` equivalent in F-Strings), it shifts to State 263 to begin parsing a nested C expression.
3. If the lookahead is `STRING_LITERAL_PART`, it shifts to State 264 to consume more standard string characters.
4. Once an `interpolation_block` is fully reduced by a later state, the parser will use the goto transition to move to State 265.

Note: The complete graphical representation of this automaton is not available since the number of states is almost 500 and visualizing it is infeasible.

6.1.3 Conflict Identification

Analyzing the LALR(1) automaton reveals exactly 3 Shift/Reduce conflicts and 0 Reduce/Reduce conflicts. Below is the explicit breakdown of each conflict, the states where they occur, and the exact productions involved. Noticeably, these conflicts existed in the original YAPL language and still exists in our extension.

```
1 State 27 conflicts: 1 shift/reduce
2 State 344 conflicts: 1 shift/reduce
3 State 466 conflicts: 1 shift/reduce
```

1. ATOMIC Specifier/Qualifier Ambiguity

```
1 State 27
2
3 165 atomic_type_specifier: ATOMIC . '(' type_name ')'
4 169 type_qualifier: ATOMIC .
5
6 '(' shift, and go to state 49
7
8 '(' [reduce using rule 169 (type_qualifier)]
9 $default reduce using rule 169 (type_qualifier)
```

(a) State: 27

(b) Type of Conflict: Shift/Reduce

(c) Productions Involved:

- Shift: `atomic_type_specifier : ATOMIC . '(' type_name ')'` (Rule 165)
- Reduce: `type_qualifier : ATOMIC .` (Rule 169)

(d) Explanation: When the parser encounters the `ATOMIC` keyword and peeks at the next token `(`, it faces an ambiguity. It does not know whether to shift the `(` to build an `atomic_type_specifier` (e.g., `_Atomic(int)`), or to immediately reduce `ATOMIC` into a `type_qualifier` (e.g., treating it like a standard qualifier in `_Atomic int x;`).

2. Mid-Rule Action (Epsilon) Ambiguity

```
1 State 344
2
3 175 declarator: pointer . $@1 direct_declarator
4 208 abstract_declarator: pointer . direct_abstract_declarator
5 209 | pointer .
6
7 '(' shift, and go to state 188
8 '[' shift, and go to state 189
9
10 IDENTIFIER reduce using rule 174 ($@1)
11 '(' [reduce using rule 174 ($@1)]
12 $default reduce using rule 209 (abstract_declarator)
13
14 $@1 go to state 136
15 direct_abstract_declarator go to state 315
```

- (a) State: 344
- (b) Type of Conflict: Shift/Reduce
- (c) Productions Involved:
 - Shift: `abstract_declarator : pointer . direct_abstract_declarator` (Rule 208)
 - Reduce: `declarator : pointer . @$1 direct_declarator` (Rule 175)
- (d) Explanation: This conflict arises from an embedded semantic action (`{pointer_decls++;}`) inside the declarator rule. Yacc translates mid-rule actions into invisible, empty (epsilon) rules (represented here as `@$1`). When the parser sits at the pointer and sees a lookahead like `(`, it must decide whether to reduce the empty rule to execute the counter (committing to a declarator), or to shift the `(` to continue building an `abstract_declarator`.

3. Dangling Else Ambiguity

```

1 State 466
2
3 263 selection_statement: IF '(' expression ')' statement . ELSE
4   $@2 statement
5 264                      | IF '(' expression ')' statement .
6
7   ELSE shift, and go to state 481
8
9   ELSE [reduce using rule 264 (selection_statement)]
   $default reduce using rule 264 (selection_statement)

```

- (a) State: 466
- (b) Type of Conflict: Shift/Reduce
- (c) Productions Involved:
 - Shift: `selection_statement : IF '(' expression ')' statement . ELSE @$2 statement` (Rule 263)
 - Reduce: `selection_statement : IF '(' expression ')' statement .` (Rule 264)
- (d) Explanation: This is the standard dangling-else ambiguity inherent in C-like grammars. When the parser completes an IF block and sees an ELSE token on the lookahead, it cannot structurally determine whether to shift the ELSE (attaching it to the closest/inner IF) or to reduce the current rule (closing the inner statement and attaching the ELSE to an outer IF).

6.2 Conflict Resolution

6.2.1 Grammar Modifications & Architecture Overhaul

While analyzing the initial Shift/Reduce conflicts—specifically the ambiguity surrounding the `ATOMIC` keyword in State 27—we made a critical architectural realization. The `ATOMIC` keyword, along with several other K&R and C99 legacy structures, were not part of the intended YAPL-S language specification. The presence of rules for these unsupported constructs was artificially bloating the parser and generating “ghost conflicts.”

Consequently, we executed a major architectural overhaul.

Major Grammar Changes

Instead of forcing a resolution on the `ATOMIC` conflict, we traced every unsupported token (e.g., `_Atomic`, `restrict`, legacy K&R function declarations) through the grammar and systematically excised their corresponding productions from both `yapl_s_new.l` and `yapl_s_new.y`.

While the sheer volume of these targeted removals is too extensive to list line-by-line, the result was a dramatic optimization of the LALR(1) automaton. The total number of states was reduced from approximately 500 down to roughly 300, permanently eliminating the State 27 conflict by completely removing the ambiguous branches.

Resolving the Mid-Rule Action

For legitimate conflicts involving supported constructs, we applied structural grammar modifications. To fix the epsilon (ϵ) conflict in the `declarator` rule (State 344), we delayed the execution of the semantic action by moving it to the end of the production.

```
1 /* BEFORE: declarator: pointer {pointer_decls++;} direct_declarator */
2
3 /* AFTER: Semantic action deferred to the end */
4 declarator
5     : pointer direct_declarator {pointer_decls++;}
6     | direct_declarator
7     ;
```

Resolving the Dangling-Else

To resolve the inherent Dangling-Else ambiguity without rewriting the core structure of C-style selection statements, we defined explicit token precedences to force the parser’s hand.

```
1 /* Precedence rules */
2 %nonassoc LOWER_THAN_ELSE
3 %nonassoc ELSE
4
5 /* Applied to the grammar */
6 selection_statement
7     : IF '(' expression ')' statement ELSE statement
8     {
9         TRACE_REDUCE("selection_statement -> IF '(' expression ')'
10         statement ELSE statement");
11     }
12     | if_without_else
13     {
14         TRACE_REDUCE("selection_statement -> if_without_else");
15     }
16     | SWITCH '(' expression ')' statement
17     {
```

```

17     TRACE_REDUCE("selection_statement -> SWITCH '(' expression
    '),' statement");
18     }
19     ;
20
21 if_without_else
22     : IF '(' expression ')' statement %prec LOWER_THAN_ELSE
23     {
24         TRACE_REDUCE("if_without_else -> IF '(' expression ')'
    statement");
25     }
26     ;

```

Re-compiling the overhauled grammar with `yacc -v -d yapl_s_new.y` yields exactly **0 conflicts** and a highly optimized parsing table.

6.2.2 Adversarial Testing of Conflicts

When LALR(1) conflicts exist, Yacc employs default heuristics to forcefully resolve the ambiguity and generate executable code (typically defaulting to a `shift` action during a Shift/Reduce conflict). To verify our grammar modifications, we compiled and executed both the original ambiguous grammar and our overhauled deterministic grammar against three specific adversarial edge cases.

The ATOMIC Ambiguity

```

1 int main() {
2     /* Should evaluate the State 27 ATOMIC ambiguity in old grammar */
3     _Atomic(int) counter;
4     return 0;
5 }

```

- **Original Compiler:** The original `yapl_s.y` triggers a Shift/Reduce conflict for the `ATOMIC` keyword. However, upon executing the adversarial code, the parser immediately throws a `syntax error, unexpected IDENTIFIER`. This reveals a fundamental desynchronization between the provided Lexer and Parser: the lexer lacked a definition for `_Atomic`, meaning the `ATOMIC` token was mathematically unreachable. The conflict was a "Ghost Conflict" existing in dead code.
- **New Compiler:** Realizing that the conflict was unreachable and the feature was unsupported by the YAPL-S specification, we completely amputated the `ATOMIC` productions from the Yacc grammar. The new compiler cleanly catches `_Atomic` as an unsupported identifier at the lexer level and rejects it.

Mid-Rule Action Ambiguity

```

1 int main() {
2     /* Triggers the State 344 epsilon conflict */
3     int (*function_ptr)(int, int);
4     return 0;
5 }

```

- **Original Compiler:** The parser successfully generated an AST (`***parsing successful***`). However, it only achieved this because Yacc's internal conflict-resolution heuristic defaults

to a **shift** action when encountering the ambiguous ϵ -transition caused by the mid-rule C-code. The success was a byproduct of toolchain guessing at generation time, not grammar accuracy.

- **New Compiler:** The parser successfully generated the exact same AST, but with **zero Shift/Reduce conflicts during parser generation**. By relocating the semantic action to the end of the production, the parser shifted the (token deterministically, proving the state machine is mathematically sound without relying on fallback heuristics.

Dangling-Else Ambiguity

```
1 int main() {
2     int x = 1, y = 1, z = 0;
3
4     /* Triggers the State 466 Dangling-Else conflict */
5     if (x > 0)
6         if (y > 0)
7             z = 1;
8     else
9         z = 2;
10
11     return 0;
12 }
```

- **Original Compiler:** The execution output yields *****parsing successful*****. The parser correctly bounds the **else** block to the innermost **if**. However, this runtime success masks an underlying generation-time ambiguity: the original grammar produced a Shift/Reduce conflict during the **yacc** build phase. The correct AST was only constructed because Yacc's hardcoded fallback heuristic always defaults to a **shift** operation, which coincidentally aligns with standard C scoping rules.
- **New Compiler:** The execution output identically yields *****parsing successful*****. The crucial architectural difference lies in the parser generation phase: the new grammar builds with **zero Shift/Reduce conflicts**. By explicitly applying the `%prec LOWER_THAN_ELSE` directive, the shift action is mathematically codified into the LALR(1) state machine. The parser structurally guarantees the correct C scoping rules without relying on any silent toolchain heuristics or default guesses.

6.3 Reverse Derivation Tree

Bottom-up parsers, including LALR(1) parsers generated by Yacc, operate by shifting input tokens onto a stack and reducing them to non-terminals when a right-hand side production is matched. The chronological sequence of these **reduce** operations corresponds to a **Rightmost Derivation in reverse**. To output the standard top-down derivation tree (from root to leaves), the parser must record every reduction and invert the sequence upon completion.

To transform the flat, 300-rule bottom-up output into a professional, hierarchically ordered Abstract Syntax Tree (AST), we implemented a multi-stage data pipeline across the Lexer (`yapl_s.1`) and Parser (`yapl_s_new.y`).

6.3.1 Lexer Changes

By default, the global `yytext` buffer is continuously overwritten as the Lexer scans new tokens. Because parser reductions occur asynchronously (often several tokens *after* a lexeme is scanned), directly printing `yytext` during a reduction yields incorrect data. To preserve the original source code mapping, we renamed the Lex scanning routine to `real_yylex()` and injected a custom wrapper function inside Yacc:

```
1 /* Interceptor Wrapper */
2 int yylex(void) {
3     int tok = real_yylex();
4     if (tok > 0 && yytext != NULL) {
5         lexeme_list[lexeme_count++] = sanitize_for_dot(yytext);
6     }
7     return tok;
8 }
```

This safely queues every exact lexeme scanned into an ordered array before the token is consumed by the parser.

6.3.2 Parser Changes

Instead of relying on `printf` logs, the `TRACE.REDUCE` macro was upgraded to store the raw string of every triggered rule into a dynamic array (`derivation_tree`).

Upon a successful parse, we iterate through this array in reverse order. We tokenize the left-hand and right-hand sides of the stored rules and dynamically allocate `TreeNode` structs. To handle Yacc's hidden mid-rule actions (which create dummy ϵ -transitions that normally desynchronize stack pops), we implemented a stack matching algorithm. Rather than blindly popping the stack, the algorithm searches the unexpanded non-terminal stack for exact string matches, cleanly ignoring phantom Yacc nodes and preserving the structural integrity of the tree.

6.3.3 Graphviz Integration

To ensure the resulting tree was human-readable, the AST is exported directly into the `.dot` language for Graphviz rendering.

- **Strict Horizontal Ordering:** Graphviz natively tangles deep trees to balance aesthetic weight. We forced a strict left-to-right code mapping by applying `ordering="out"` and using invisible structural edges (`-> [style=invis]`) inside a `{ rank=same; }` block to lock all terminal leaves to the bottom baseline.
- **Post-Processing:** Before export, a recursive function traverses the leaves from left to right. It actively filters out unexpanded ϵ -rules (Phantom Leaves) by detecting lower-case non-terminal identifiers. It then attaches the saved source-code strings from the



The result is a mathematically complete, strictly ordered, high-resolution SVG visual map of the program’s rightmost derivation.

```
1 int number = 10;
2 char *example = f"Hello. The number is {number}. " @ "This is so easy.";
```

6.4 LALR(1) Parsing Table

To generate LALR(1) Parsing Table, we again relied on the verbose output files generated in Section 6.1, carefully parsing the shift, reduce and goto actions for each state.

A partial comparison of the resulting LALR(1) tables is shown in Figure 2. Figure 2a visualizes the original Shift/Reduce conflict in State 27 of the old grammar, which was the primary catalyst for the re-design. Figure 2b shows a successful segment of the new, optimized parsing table, which is entirely conflict-free.

The complete parsing tables for both the original and new grammar are attached to this report as standalone HTML files, `parsing_table_old.html` and `parsing_table_new.html`. We recommend viewing these files in a web browser.

[illegible]

(a) Conflict in State 27 of Old Grammar
(yap1_s)

[illegible]

(b) Conflict-Free Portion of New Grammar
(yap1_s_new) Parsing Table

Figure 2: Comparison of original parsing conflict versus new deterministic table.

6.5 Error Diagnosis

By default, standard Yacc implementations halt immediately upon encountering an invalid token sequence and emit a generic, unhelpful string: **syntax error**. In addition to this, we implemented code to pinpoint the exact spatial location and grammatical nature of a syntax violation.

The diagnostic system required modifications across both the Lexer and the Parser.

1. **Lexer Changes:** We enabled `%option yylineno` within `Lex` to automatically track the active line number. However, column tracking requires manual intervention. To accurately compute column numbers across multi-line comments and whitespace blocks, we defined a custom `YY_USER_ACTION` macro. This macro intercepts the lexer prior to executing any token’s semantic action, iterates through the matched `yytext` buffer, and dynamically advances or resets a global `yycolumn` counter.
2. **Verbose State Exposure:** We injected the `%define parse.error verbose` directive into the Yacc specification. Instead of simply failing, this directive instructs the LALR(1) automaton to cross-reference its current state against the lookahead table upon a failure, allowing it to explicitly list the expected tokens versus the offending token.

To demonstrate the diagnostic enhancement, we fed both the original and modified compilers a C program with a fault.

Example Program:

```
1 int main() {
2     int x = 10;
3     // Missing the closing '}' for the interpolation block
4     char *broken = f"The value is { x + 5 ";
5
6     return 0;
7 }
```

Original Compiler Output:

```
1 ***parsing terminated*** [syntax error]
```

New YAPL-S Compiler Output

```
1 =====
2                               COMPILER ERROR
3                               =====
4 Location : Line 4, Column 44
5 Token    : ';'
6 Message  : syntax error, unexpected ';', expecting INTERPOLATION_END or
7           ', '
8 ***parsing terminated*** [syntax error]
```

6.6 Shared Artifacts

Along with this report, we have provided the complete source code repository for the YAPL-S: Github Repo

Using the provided Makefile, one can generate both the original and updated lexers and parsers to test the behavioral differences. Detailed build instructions and execution commands are available in the dedicated README.md file.

The repository is organized into the following key components:

- `yapl_s.l` and `yapl_s.y`: The original, unoptimized Lex and Yacc files containing the unsupported legacy constructs and documented Shift/Reduce conflicts.
- `yapl_s_new.l` and `yapl_s_new.y`: The overhauled, strictly deterministic grammar files.
- `parser_analysis/`: This includes the `generate_table.py` script used to construct the interactive HTML parsing tables, and the `adversarial_tests/` subdirectory containing the specific C snippets and execution logs used to verify the conflict resolutions.
- `tests/`: A comprehensive evaluation suite categorized into `positive/`, `negative/`, and `misc/` directories. Each subdirectory contains test C files alongside side-by-side execution logs (`output_old_*.txt` vs. `output_new_*.txt`) and generated Reverse Derivation Trees in SVG format for the successfully-compiled programs using the new compiler.